Rigging System Planning

Requirements:

- Written for Maya 2026+ in the new maya api (maya.api.OpenMaya)
- Strictly follows the api for what truly exists
- has abstraction layers
- Networked and traversable
- Modular rigs
- Written in c++
- Able to be baked down to a skeleton
- Able to be reanimated from a skeleton
- Planned out beforehand
- Straightforward creation for the rigger (plugin commands)
    - Ex: pass a JSON file that's the "rig" to a plugin command (RigRoot command) which can read the JSON and create the modules
- Parent child relationships between controls are clear
- Extendable for facial rigging
- Inputs will be fairly explicit.
    - Joints, either by name or by markup requirements ("left", "upper_arm")
- Joints will not be created by the modules. It is a rig on top of joints
- Documented and commented throughout
- Controls will be created by the modules.
- Modules have clear world space sockets available for connections to other modules
    - These sockets are joints or calculated transforms
    - N number modules can be made children of a given module at a given socket.
- Controls are in a normal DAG hierarchy with inherits transforms OFF (the modules are the dag brain)
- The node acts as the motion functionality and replaces constraints and complex node groups in the outliner
- Picking is done via Controller definitions (define the custom parent\child control for up and down, left and right)
- MPlug attre should use the naming convention "pPlugName"
- Node attr mobjects should follow the name convention "aAttrName"
- control child plugs get dag parented to control parent sockets

This rigging system would be an extension of all the things I've learned as a rigging td. It would be a showcase of pipeline knowledge (reanimation, skeleton baking), speed and api knowledge (c++ modules), and mathematical knowledge (being c++ modules means the logic is matrix math).

Because it is a c++ plugin, the inputs would be matrices (pins? Joint rest positions?) and the outputs would be control and skeleton matrices. This is very similar to the Bifrost setups where you'd have double transformations become possible.

I need to think and plan this through thoroughly before beginning. At least a few modules, because I won't be able to waffle. There won't be the kind of python inheritance you usually have with these types of modules. Each one will be there own type of node.

A module node would be created by calling a maya.cmds plugin call to create the node. The node and the plugin command that creates it will be the same (no special commands for creating the node). That means the initializer for the node takes all methods for creating it.

A module will have a parent module, or else it will be the rig root.

Module attributes will be stamped on a json string for parsing.

The rest structure of the module will be based on input matrices. These can come from the rest matrices of the input joints. For special pivots like the reverse foot, null joints can be created.

X will be the main down axis. With +Z the bend.

The simplest single fk module will just take a joint as an input and create a control that is the parent (matrix multiplication: control parent inverse matrix × rest world matrix × control world matrix ??)

Node Structure:

- Asset Root
    - Track the high level asset information like its ID, type, etc. Is the root of the asset used to identify it
    - Attributes are stamped onto it in json format as native Maya string attributes
- Rig Root (the layout node)
- Module Nodes

Maya C++ MMatrix Order Rules:

- **Row-Major Post-Multiplication Convention:** Maya's MMatrix post-multiplies transformations. To transform a child offset into a parent's world space, the formula is:
  $$\text{World Matrix} = \text{Local Step} \times \text{Parent World Matrix}$$
- **Local Step Composition:** Each step along a module chain combines local control channels with static rest offsets:
  $$\text{Local Step}_i = \text{Control Local}_i \times \text{Rest Matrix}_i$$
- **Array Iteration Accumulation (Flat Hierarchy Solve):** For a module evaluating index $0 \dots N$ in an array, the sequential accumulation loop formula is:
  $$\mathbf{M}_{\text{SocketWorld}_i} = \mathbf{M}_{\text{Control}_i} \times \mathbf{M}_{\text{Rest}_i} \times \mathbf{M}_{\text{SocketWorld}_{i-1}}$$

(Where $\mathbf{M}_{\text{SocketWorld}_{-1}}$ is the incoming parentWorldMatrix from the upstream module socket).

You are using maya's new api and Maya 2026

Never put curly brackets in line, always start a new line
Example:

```
struct FkModuleArgs
{
   std::string shapeType = "Cube";
   bool addGimbalControls = false;
};
```

Not

```
struct FkModuleArgs {
   std::string shapeType = "Cube";
   bool addGimbalControls = false;
};
```

1. Prefer In-Place Array Mutation Over MArrayDataBuilder in Evaluation Contexts

- **The Pitfall:** Creating a new MArrayDataBuilder inside compute() / evaluate() and overwriting the output handle via hOutSocketArray.set(builder) destroys existing datablock handle mappings. This causes downstream nodes connected to specific array indices (e.g., outputSocketMatrix[0]) to lose their evaluation state, silently failing index jumps (jumpToElement) or preventing Maya from triggering evaluation altogether.
- **The Solution:** Use MArrayDataBuilder strictly during node initialization or dynamic attribute resizing. During solver evaluation, perform direct, in-place updates on existing datablock element handles via hOutSocketArray.jumpToElement(index) and hOutSocketArray.outputValue().setMMatrix(...).

2. Sequential Accumulation & Complete Array Cleaning

- **The Rule:** In custom hierarchical solvers where child transforms depend on parent steps ($M_{SocketWorld_i} = M_{Control_i} \times M_{Rest_i} \times M_{SocketWorld_{i-1}}$), the solver must iterate through the entire input array sequentially from $0 \dots N$.
- **Clean State:** After writing all accumulated matrices to the output array handles in-place, mark the entire handle clean at once using hOutSocketArray.setAllClean() and data.setClean(plug)